gapgpt_transcriber.py

```python
import os
import queue
import tempfile
import threading
import wave
from pathlib import Path

import requests
import sounddevice as sd
import tkinter as tk
from dotenv import load_dotenv
from tkinter import filedialog, messagebox, ttk


DEFAULT_BASE_URL = "https://api.gapgpt.app/v1"
TRANSCRIPT_PATH = "/audio/transcriptions"
DEFAULT_MODEL = "gapgpt/whisper-1"
ENV_FILE = ".env"


class GapGPTTranscriberApp:
    def __init__(self, root: tk.Tk) -> None:
        self.root = root
        self.root.title("GapGPT Voice Transcriber")
        self.root.geometry("980x700")

        load_dotenv(ENV_FILE)

        self.api_key_var = tk.StringVar(value=os.getenv("GAPGPT_API_KEY", ""))
        self.base_url_var = tk.StringVar(
            value=os.getenv("GAPGPT_BASE_URL", DEFAULT_BASE_URL)
        )
        self.model_var = tk.StringVar(value=os.getenv("GAPGPT_MODEL", DEFAULT_MODEL))
        self.sample_rate_var = tk.StringVar(
            value=os.getenv("AUDIO_SAMPLE_RATE", "16000")
        )
        self.channels_var = tk.StringVar(value=os.getenv("AUDIO_CHANNELS", "1"))
        self.status_var = tk.StringVar(value="Idle")
        self.error_var = tk.StringVar(value="")

        self.is_recording = False
        self.audio_chunks = []
        self.audio_queue = queue.Queue()
        self.stream = None

        self._build_ui()
        self.root.protocol("WM_DELETE_WINDOW", self.on_close)
```

```
48
49      def _build_ui(self) -> None:
50          self.root.rowconfigure(1, weight=1)
51          self.root.columnconfigure(0, weight=1)
52
53          top_frame = ttk.Frame(self.root, padding=10)
54          top_frame.grid(row=0, column=0, sticky="nsew")
55          top_frame.columnconfigure(1, weight=1)
56
57          ttk.Label(top_frame, text="API Key").grid(
58              row=0, column=0, sticky="w", padx=(0, 8), pady=4
59          )
60          ttk.Entry(top_frame, textvariable=self.api_key_var, show="*").grid(
61              row=0, column=1, sticky="ew", pady=4
62          )
63
64          ttk.Label(top_frame, text="Base URL").grid(
65              row=1, column=0, sticky="w", padx=(0, 8), pady=4
66          )
67          ttk.Entry(top_frame, textvariable=self.base_url_var).grid(
68              row=1, column=1, sticky="ew", pady=4
69          )
70
71          ttk.Label(top_frame, text="Model").grid(
72              row=2, column=0, sticky="w", padx=(0, 8), pady=4
73          )
74          ttk.Entry(top_frame, textvariable=self.model_var).grid(
75              row=2, column=1, sticky="ew", pady=4
76          )
77
78          audio_frame = ttk.Frame(top_frame)
79          audio_frame.grid(row=3, column=0, columnspan=2, sticky="w", pady=4)
80
81          ttk.Label(audio_frame, text="Sample Rate").grid(row=0, column=0, sticky="w")
82          ttk.Entry(audio_frame, textvariable=self.sample_rate_var, width=10).grid(
83              row=0, column=1, sticky="w", padx=(8, 20)
84          )
85
86          ttk.Label(audio_frame, text="Channels").grid(row=0, column=2, sticky="w")
87          ttk.Entry(audio_frame, textvariable=self.channels_var, width=10).grid(
88              row=0, column=3, sticky="w", padx=(8, 0)
89          )
90
91          button_frame = ttk.Frame(top_frame)
92          button_frame.grid(row=4, column=0, columnspan=2, sticky="w", pady=(10, 6))
93
94          self.start_button = ttk.Button(
95              button_frame, text="Start Recording", command=self.start_recording
96          )
```

```
        self.start_button.grid(row=0, column=0, padx=(0, 8))

        self.stop_button = ttk.Button(
            button_frame,
            text="Stop Recording",
            command=self.stop_recording,
            state="disabled",
        )
        self.stop_button.grid(row=0, column=1, padx=(0, 8))

        ttk.Button(button_frame, text="Copy All", command=self.copy_all).grid(
            row=0, column=2, padx=(0, 8)
        )
        ttk.Button(button_frame, text="Save Text", command=self.save_text).grid(
            row=0, column=3, padx=(0, 8)
        )
        ttk.Button(button_frame, text="Clear", command=self.clear_text).grid(
            row=0, column=4, padx=(0, 8)
        )
        ttk.Button(button_frame, text="Save Settings", command=self.save_settings).grid(
            row=0, column=5, padx=(0, 8)
        )

        status_frame = ttk.Frame(top_frame)
        status_frame.grid(row=5, column=0, columnspan=2, sticky="ew", pady=(4, 0))
        status_frame.columnconfigure(1, weight=1)

        ttk.Label(status_frame, text="Status").grid(
            row=0, column=0, sticky="nw", padx=(0, 8)
        )
        ttk.Label(status_frame, textvariable=self.status_var).grid(
            row=0, column=1, sticky="nw"
        )

        ttk.Label(status_frame, text="Error").grid(
            row=1, column=0, sticky="nw", padx=(0, 8), pady=(4, 0)
        )
        ttk.Label(
            status_frame,
            textvariable=self.error_var,
            foreground="red",
            justify="left",
            wraplength=800,
        ).grid(row=1, column=1, sticky="ew", pady=(4, 0))

        bottom_frame = ttk.Frame(self.root, padding=(10, 0, 10, 10))
        bottom_frame.grid(row=1, column=0, sticky="nsew")
        bottom_frame.rowconfigure(0, weight=1)
        bottom_frame.columnconfigure(0, weight=1)
```

```
146
147         self.text_widget = tk.Text(bottom_frame, wrap="word", undo=True)
148         self.text_widget.grid(row=0, column=0, sticky="nsew")
149
150         scrollbar = ttk.Scrollbar(
151             bottom_frame, orient="vertical", command=self.text_widget.yview
152         )
153         scrollbar.grid(row=0, column=1, sticky="ns")
154         self.text_widget.configure(yscrollcommand=scrollbar.set)
155
156     def set_status(self, text: str) -> None:
157         self.status_var.set(text)
158         self.root.update_idletasks()
159
160     def set_error(self, text: str) -> None:
161         self.error_var.set(text)
162         self.root.update_idletasks()
163
164     def clear_error(self) -> None:
165         self.set_error("")
166
167     def audio_callback(self, indata, frames, time_info, status) -> None:
168         if status:
169             self.audio_queue.put(("error", str(status)))
170         self.audio_queue.put(("audio", indata.copy()))
171
172     def start_recording(self) -> None:
173         if self.is_recording:
174             return
175
176         api_key = self.api_key_var.get().strip()
177         if not api_key:
178             messagebox.showerror(
179                 "Missing API Key",
180                 "Set GAPGPT_API_KEY in .env or enter it in the app.",
181             )
182             return
183
184         try:
185             sample_rate = int(self.sample_rate_var.get().strip())
186             channels = int(self.channels_var.get().strip())
187         except ValueError:
188             messagebox.showerror(
189                 "Invalid Audio Settings",
190                 "Sample rate and channels must be integers.",
191             )
192             return
193
194         self.audio_chunks = []
```

```
195         self.audio_queue = queue.Queue()
196         self.clear_error()
197 
198         try:
199             self.stream = sd.InputStream(
200                 samplerate=sample_rate,
201                 channels=channels,
202                 dtype="int16",
203                 callback=self.audio_callback,
204             )
205             self.stream.start()
206         except Exception as exc:
207             self.set_error(f"Failed to start recording: {exc}")
208             self.set_status("Recording failed")
209             return
210 
211         self.is_recording = True
212         self.start_button.config(state="disabled")
213         self.stop_button.config(state="normal")
214         self.set_status("Recording...")
215         self.root.after(100, self.process_audio_queue)
216 
217     def process_audio_queue(self) -> None:
218         while not self.audio_queue.empty():
219             item_type, payload = self.audio_queue.get()
220             if item_type == "error":
221                 self.set_error(payload)
222             elif item_type == "audio":
223                 self.audio_chunks.append(payload)
224 
225         if self.is_recording:
226             self.root.after(100, self.process_audio_queue)
227 
228     def stop_recording(self) -> None:
229         if not self.is_recording:
230             return
231 
232         self.is_recording = False
233 
234         try:
235             if self.stream is not None:
236                 self.stream.stop()
237                 self.stream.close()
238                 self.stream = None
239         except Exception as exc:
240             self.set_error(f"Failed to stop recording cleanly: {exc}")
241 
242         while not self.audio_queue.empty():
243             item_type, payload = self.audio_queue.get()
```

```
            if item_type == "error":
                self.set_error(payload)
            elif item_type == "audio":
                self.audio_chunks.append(payload)

        self.start_button.config(state="normal")
        self.stop_button.config(state="disabled")

        if not self.audio_chunks:
            self.set_status("No audio captured")
            return

        self.set_status("Saving WAV and uploading...")
        threading.Thread(target=self.save_and_transcribe, daemon=True).start()

    def save_and_transcribe(self) -> None:
        temp_wav = None

        try:
            import numpy as np

            sample_rate = int(self.sample_rate_var.get().strip())
            channels = int(self.channels_var.get().strip())

            audio_data = np.concatenate(self.audio_chunks, axis=0)

            with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as wav_file:
                temp_wav = wav_file.name

            with wave.open(temp_wav, "wb") as wav_writer:
                wav_writer.setnchannels(channels)
                wav_writer.setsampwidth(2)
                wav_writer.setframerate(sample_rate)
                wav_writer.writeframes(audio_data.tobytes())

            transcript = self.send_transcription(temp_wav, "audio/wav")
            self.root.after(0, self.append_transcript, transcript)
            self.root.after(0, self.set_status, "Transcription complete")
            self.root.after(0, self.clear_error)
        except Exception as exc:
            self.root.after(0, self.set_error, f"Transcription failed: {exc}")
            self.root.after(0, self.set_status, "Error")
        finally:
            if temp_wav and Path(temp_wav).exists():
                try:
                    Path(temp_wav).unlink()
                except OSError:
                    pass

```

```python
    def send_transcription(self, file_path: str, mime_type: str) -> str:
        base_url = self.base_url_var.get().strip().rstrip("/")
        api_key = self.api_key_var.get().strip()
        model = self.model_var.get().strip()

        if not base_url:
            raise ValueError("Base URL is empty.")
        if not api_key:
            raise ValueError("API key is empty.")
        if not model:
            raise ValueError("Model is empty.")

        url = f"{base_url}{TRANSCRIPT_PATH}"
        headers = {"Authorization": f"Bearer {api_key}"}

        with open(file_path, "rb") as file_obj:
            files = {"file": (Path(file_path).name, file_obj, mime_type)}
            data = {
                "model": model,
                "response_format": "text",
            }
            response = requests.post(
                url,
                headers=headers,
                files=files,
                data=data,
                timeout=300,
            )

        if not response.ok:
            raise RuntimeError(f"{response.status_code} {response.text}")

        content_type = response.headers.get("content-type", "").lower()
        if "application/json" in content_type:
            payload = response.json()
            if isinstance(payload, dict):
                return payload.get("text") or payload.get("output_text") or str(payload)
            return str(payload)

        return response.text.strip()

    def append_transcript(self, transcript: str) -> None:
        if not transcript.strip():
            transcript = "[Empty transcription response]"

        existing = self.text_widget.get("1.0", "end-1c").strip()
        if existing:
            self.text_widget.insert("end", "\n\n")
        self.text_widget.insert("end", transcript)
```

```
342         self.text_widget.see("end")
343
344     def copy_all(self) -> None:
345         text = self.text_widget.get("1.0", "end-1c")
346         self.root.clipboard_clear()
347         self.root.clipboard_append(text)
348         self.set_status("Copied to clipboard")
349
350     def save_text(self) -> None:
351         text = self.text_widget.get("1.0", "end-1c")
352         file_path = filedialog.asksaveasfilename(
353             title="Save transcription",
354             defaultextension=".txt",
355             filetypes=[("Text files", "*.txt"), ("All files", "*.*")],
356         )
357         if not file_path:
358             return
359
360         Path(file_path).write_text(text, encoding="utf-8")
361         self.set_status(f"Saved to {file_path}")
362
363     def clear_text(self) -> None:
364         self.text_widget.delete("1.0", "end")
365         self.set_status("Cleared")
366
367     def save_settings(self) -> None:
368         env_text = (
369             f'GAPGPT_API_KEY="{self.api_key_var.get().strip()}"\n'
370             f'GAPGPT_BASE_URL="{self.base_url_var.get().strip()}"\n'
371             f'GAPGPT_MODEL="{self.model_var.get().strip()}"\n'
372             f'AUDIO_SAMPLE_RATE="{self.sample_rate_var.get().strip()}"\n'
373             f'AUDIO_CHANNELS="{self.channels_var.get().strip()}"\n'
374         )
375         Path(ENV_FILE).write_text(env_text, encoding="utf-8")
376         self.set_status(f"Saved settings to {ENV_FILE}")
377
378     def on_close(self) -> None:
379         if self.is_recording:
380             try:
381                 self.stop_recording()
382             except Exception:
383                 pass
384         self.root.destroy()
385
386
387 def main() -> None:
388     root = tk.Tk()
389     ttk.Style().theme_use("clam")
390     GapGPTTranscriberApp(root)
```

```
    root.mainloop()


if __name__ == "__main__":
    main()

```